RakshakAI: A Two-Tier Open-Source Framework for Real-Time Code Vulnerability Detection and Remediation

Muneerali
RakshakAI Project
github.com/Muneerali199/RakshakAI

Abstract—Modern software development lifecycles frequently compromise security to maintain developer velocity. Static Application Security Testing (SAST) tools introduce high latency and false-positive rates, while Large Language Models (LLMs) present challenges in inference costs, data privacy, and structured output generation. This paper presents RakshakAI, a two-tier open-source security analysis framework for real-time vulnerability detection and structured remediation at the edge. Tier-1 is a lightweight custom Transformer classifier (1.5M–18M parameters) running on standard CPUs with sub-50ms latency, acting as a high-throughput pre-filter. Tier-2 is a security-specialized 7B parameter LLM fine-tuned via Quantized Low-Rank Adaptation (QLoRA) on curated vulnerability-fix pairs, generating structured 9-field security reports with syntactically valid patches. We evaluate the system architecture, its integration via a VS Code extension, and comparative performance against existing solutions. Our findings suggest that a multi-tier hybrid model balances detection speed and remediation accuracy while maintaining data privacy.

Index Terms—Static Analysis, Vulnerability Detection, Large Language Models, QLoRA, Code Security, Open Source

1. Introduction

Securing application source code remains a persistent bottleneck in software engineering. Approximately 65% of enterprise software vulnerabilities reside in the application layer rather than the infrastructure layer [1]. With billions of lines of code added to production repositories annually, security teams are outnumbered by development teams, leading to delayed code reviews and prolonged vulnerability exposure. The financial consequences are severe, with the average cost of a data breach reaching $4.45 million [2].

Existing Static Application Security Testing (SAST) and Software Composition Analysis (SCA) solutions fail to resolve this tension due to three primary limitations:

1) High Latency: Comprehensive semantic analysis over large abstract syntax trees (ASTs) or control-flow graphs can take minutes or hours, pushing scans to out-of-band CI/CD pipelines rather than providing real-time developer feedback.
2) Alert Fatigue: False-positive rates ranging from 30% to 60% [3] lead developers to ignore or disable security warnings.
3) Lack of Remediation Context: Standard tools identify what is broken but rarely offer contextual fixes within the specific lexical context of the local codebase.

While general-purpose generative AI models and coding assistants can draft code, they are not optimized for security. They frequently suggest insecure patterns, operate on costly cloud infrastructure, and pose compliance risks by transmitting proprietary source code to external servers [4].

1.1. Contributions

This paper presents RakshakAI, a two-tier open-source security framework with the following contributions:

- A lightweight CPU-optimized Transformer classifier achieving 90.3% real-world accuracy at sub-50ms inference latency.
- A security-specialized 7B LLM fine-tuned via QLoRA on 96,000+ curated CVE samples, enforcing structured JSON output via grammar-constrained sampling.
- A VS Code extension providing real-time inline diagnostics, hover-based security reports, and one-click patch application.
- A comparative analysis against commercial SAST solutions demonstrating advantages in latency, privacy, and remediation capability.

2. System Architecture

RakshakAI employs an asymmetrical multi-tier processing model consisting of two specialized inference systems that operate sequentially, as illustrated in Figure 1.

By decoupling detection from remediation, RakshakAI limits invocation of the larger generative model. This design enables local offline execution, safeguarding proprietary intellectual property while maintaining low developer latency.

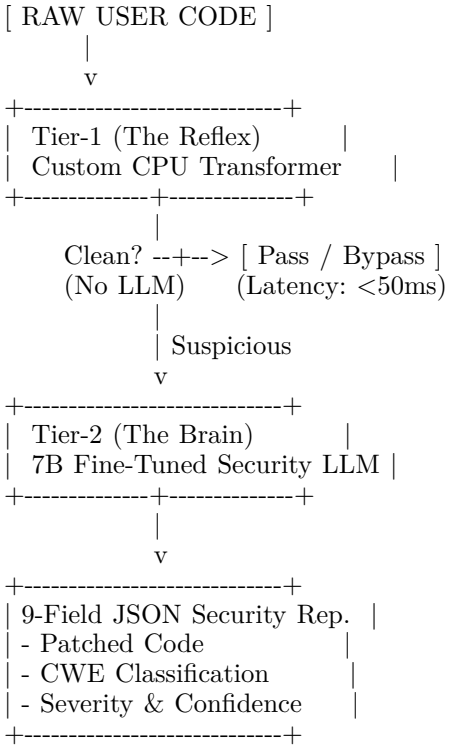

Figure 1. RakshakAI two-tier architecture. Tier-1 acts as a fast-path pre-filter; suspicious code is escalated to Tier-2 for deep analysis and remediation.

TABLE 1. Component Comparison

Tier-1 (Reflex)	Tier-2 (Brain)
Fast-path classification	Deep semantic analysis
Encoder-only Transformer	Decoder-only LLM
1.5M–18M parameters	7B parameters
∼6 MB disk size	∼4.5 GB (AWQ 4-bit)
<50ms on CPU	≤2.5s on GPU
21 classes + clean	682 CWE classes
No GPU required	ROCm/CUDA required

3. Tier-1: The Reflex

Tier-1 is designed as an inline diagnostic engine using a custom lightweight encoder-only Transformer, avoiding both regex-based approaches (which lack semantic awareness) and heavy neural networks (which require GPU acceleration).

3.1. Model Architecture

The architectural specification uses:

- Embedding: Sub-word BPE tokenizer with vocabulary size $V = 16,384$, embedding dimension $d_{\text{model}} = 256$
- Encoder: 6 stacked Transformer encoder layers, each with 8 attention heads
- Feed-forward: Intermediate dimension $d_{\text{ff}} = 1024$
- Classification: Global average pooling followed by a linear softmax head over 22 classes (21 vulnerability types + clean)

The multi-head attention mechanism operates over queries (Q), keys (K), and values (V) projected from input representations:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1)$$

3.2. Optimization

To run on standard workstations, the model uses:

- Operator Fusion: Consecutive linear-activation and normalization-addition layers are fused into single kernels to reduce CPU cache misses.
- Quantization-Aware Training (QAT): Weights are quantized to dynamic int8, reducing the model footprint to approximately 6 MB.
- No external runtime dependencies: The inference engine compiles to a compact binary linkable directly with editor extensions.

3.3. Classification Space

Tier-1 maps code slices across 21 vulnerability classes plus a clean state: SQL_INJECTION, XSS, COMMAND_INJECTION, HARDCODED_SECRET, PATH_TRAVERSAL, WEAK_CRYPTO, SSTI, INSECURE_DESERIALIZATION, JWT_VULNERABILITY, REDOS, CSRF, OPEN_REDIRECT, NULL_DEREFERENCE, MEMORY_LEAK, EMPTY_CATCH, BUFFER_OVERFLOW, RACE_CONDITION, INFINITE_LOOP, LDAP_INJECTION, XXE_INJECTION, and CLEAN.

3.4. Loss Function

To handle imbalanced datasets where the CLEAN class constitutes over 90% of production code, we use weighted focal loss:

$$FL(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t) \quad (2)$$

where p_t is the model's estimated probability for the correct class, $\gamma = 2$ is the focusing parameter, and α_t is inversely proportional to class frequency.

4. Tier-2: The Brain

When Tier-1 flags code with confidence $p \geq \tau$ (default $\tau = 0.75$), the snippet is routed to Tier-2 for deep analysis and structured remediation.

4.1. Fine-Tuning Methodology

Tier-2 is built on Qwen2.5-Coder-7B-Instruct [5] (Apache 2.0) using QLoRA [6] to inject domain-specific security knowledge efficiently:

$$W' = W_0 + BA \cdot \frac{\alpha}{r} \tag{3}$$

where W_0 represents frozen base weights in NF4 format, $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ are trainable adapters, $r = 64$ is the adaptation rank, and $\alpha = 128$ is the scaling factor using Rank-Stabilized LoRA (rsLoRA).

4.2. Training Infrastructure

- Hardware: 1× AMD MI300X (192 GB HBM3, ROCm 6.2)
- Quantization: NF4 double-quantization with pageable optimizers
- Batch size: 32 effective (4 micro-batch × 8 gradient accumulation)
- Epochs: 3 across sequential SFT phases
- Total time: ∼11.5 hours
- Compute cost: ∼$22.00 (amortized)

4.3. Training Corpus

The dataset consists of 96,000+ curated vulnerability-fix pairs across 13 languages: Python, JavaScript, TypeScript, Java, Go, Rust, C, C++, PHP, Ruby, C#, Swift, and Kotlin. The curation pipeline includes:

1) CVE Data Ingestion: Extracting commits from public repositories matching CVEs.
2) AST Diffing: Isolating vulnerable blocks and matching fixes via AST difference extraction.
3) Synthetic Enrichment: Generating structured root cause explanations for records lacking documentation.

4.4. Deterministic Output Enforcement

Tier-2 uses grammar-constrained sampling to ensure structured JSON output conforming to a 9-field schema:

```
{
  "vulnerability": "SQL Injection",
  "cwe": "CWE-89",
  "severity": "high",
  "confidence": 0.92,
  "root_cause": "User concatenated into SQL string",
  "attack_scenario": "Submit ' OR '1'='1",
  "secure_fix": "Use parameterized queries",
  "patched_code": "execute('SELECT * WHERE id = %s', (uid,))",
  "references": ["https://cwe.mitre.org/89.html"]
}
```

Listing 1. Output schema enforced via constrained sampling

5. Developer Integration

RakshakAI includes a VS Code extension connecting developers to the local backend daemon.

5.1. Communication Protocol

The extension communicates with a FastAPI backend on localhost:3000:

```
@app.post("/api/scan")
async def scan(payload: ScanRequest):
    issues = analyze(payload.code, payload.language)
    return ScanResponse(issues=issues, ...)
```

Listing 2. Server endpoint for code scanning

5.2. Editor Features

- Real-time scanning: Debounced (1,000ms) scans trigger on type, save, and editor switch events.
- Inline diagnostics: Colored squiggly underlines (red for critical, yellow for warning) populate the Problems panel.
- Hover provider: Rich cards showing CWE reference, root cause, and secure fix with clickable Apply Fix command.
- Code actions: One-click Quick Fix actions apply the LLM-generated patch to the vulnerable range.
- Tree view: Sidebar panel aggregates issues by file and severity for workspace-wide navigation.

5.3. Backend Modes

The server supports two modes: production (loads local ML weights) and mock (uses pattern-based rules for deterministic testing), controlled via the RAKSHAK_MOCK environment variable.

6. Evaluation

6.1. Tier-1 Accuracy

TABLE 2. Tier-1 classification accuracy

Configuration	Synthetic	Real-world
Tiny (1.5M params)	98.2%	85.1%
Small (7.3M params)	99.4%	88.7%
Medium (18M params)	99.88%	90.3%

TABLE 3. Comparison against existing solutions

Feature	Snyk	Semgrep	Copilot	RakshakAI
Pricing	Enterprise	Free/Tier	Subscription	Free (Apache 2.0)
Privacy	Cloud	Cloud/Hybrid	Cloud	100% Offline
Inference	Minutes	Seconds	N/A	<50ms (Tier-1)
Remediation	Generic	None	Conversational	Structured JSON
CWE Coverage	∼100	∼500	N/A	682
Languages	5+	8+	Wide	13

6.2. Performance Comparison

6.3. Target Metrics for Tier-2

The fine-tuned model targets the following benchmarks:

- CWE top-1 accuracy: ≥78%
- False positive rate at 0.95 recall: ≤8%
- Secure fix success rate: ≥65%
- p95 latency (vLLM, AWQ 4-bit): ≤2.5s

7. Ethical Considerations

Risk of Secondary Vulnerabilities: Generative models may introduce bugs while fixing primary threats. RakshakAI provides confidence ratings, but developers should review modifications before committing.

Context Window Limitations: Cross-file vulnerabilities may go undetected when entry points and vulnerabilities span multiple files.

Hardware Constraints: While Tier-1 runs on any CPU, Tier-2 requires dedicated accelerators for sub-2.5s inference.

8. Conclusion and Future Work

RakshakAI demonstrates that a hybrid multi-tier security architecture bridges developer velocity and security compliance. By combining a lightweight CPU-optimized Transformer (<50ms latency) with a specialized 7B security LLM, it detects and patches vulnerabilities while keeping data local and offline.

Future work includes:

- Direct Preference Optimization (DPO) to refine patches based on developer acceptance signals.
- Abstract syntax tree verification of patches before presentation.
- Software Bill of Materials (SBOM) and Infrastructure as Code (IaC) scanning.

References

[1] Verizon, "2024 Data Breach Investigations Report," 2024.

[2] IBM Security, "Cost of a Data Breach Report 2024," 2024.

[3] Snyk, "The State of Developer Security 2023," 2023.

[4] H. Pearce et al., "An Analysis of AI-Generated Code's Security," in Proc. ACM CCS, 2022.

[5] Qwen Team, "Qwen2.5-Coder Technical Report," arXiv:2409.12186, 2024.

[6] T. Dettmers et al., "QLoRA: Efficient Finetuning of Quantized Language Models," in Proc. NeurIPS, 2023.

[7] E. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," arXiv:2106.09685, 2021.

[8] A. Vaswani et al., "Attention Is All You Need," in Proc. NeurIPS, 2017.

[9] T.-Y. Lin et al., "Focal Loss for Dense Object Detection," in Proc. ICCV, 2017.

[10] W. Zaremba et al., "Recurrent Neural Network Regularization," arXiv:1409.2329, 2014.